```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd

chunksize = 100000  # adjust based on memory
chunks = []

for chunk in pd.read_csv("/content/drive/MyDrive/US_Accidents_March23.csv", chunksize=chunksize):
    # Optional: filter or process each chunk here
    chunks.append(chunk)

# Combine chunks (if needed)
df = pd.concat(chunks, ignore_index=True)
print(df.shape)
```

```
(7728394, 46)
```

```
df.info()
df.describe(include="all")
```

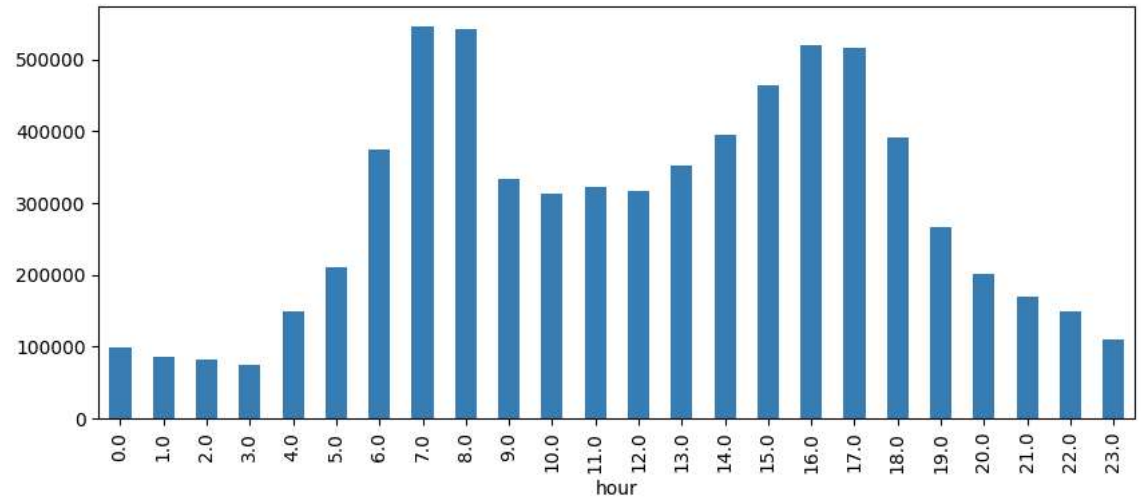
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7728394 entries, 0 to 7728393
Data columns (total 46 columns):
 #   Column                 Dtype
---  ------                 -----
 0   ID                     object
 1   Source                 object
 2   Severity               int64
 3   Start_Time             object
 4   End_Time               object
 5   Start_Lat              float64
 6   Start_Lng              float64
 7   End_Lat                float64
 8   End_Lng                float64
 9   Distance(mi)           float64
 10  Description            object
 11  Street                 object
 12  City                   object
 13  County                 object
 14  State                  object
 15  Zipcode                object
 16  Country                object
 17  Timezone               object
 18  Airport_Code           object
 19  Weather_Timestamp      object
 20  Temperature(F)         float64
 21  Wind_Chill(F)          float64
 22  Humidity(%)            float64
 23  Pressure(in)           float64
 24  Visibility(mi)         float64
 25  Wind_Direction         object
 26  Wind_Speed(mph)        float64
 27  Precipitation(in)      float64
 28  Weather_Condition      object
 29  Amenity                bool
 30  Bump                   bool
 31  Crossing               bool
 32  Give_Way               bool
 33  Junction               bool
 34  No_Exit                bool
 35  Railway                bool
 36  Roundabout             bool
 37  Station                bool
 38  Stop                   bool
 39  Traffic_Calming        bool
 40  Traffic_Signal         bool
 41  Turning_Loop           bool
 42  Sunrise_Sunset         object
 43  Civil_Twilight         object
 44  Nautical_Twilight      object
 45  Astronomical_Twilight  object
dtypes: bool(13), float64(12), int64(1), object(20)
memory usage: 2.0+ GB
```

	ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng	Distanc
count	7728394	7728394	7.728394e+06	7728394	7728394	7.728394e+06	7.728394e+06	4.325632e+06	4.325632e+06	7.72839
unique	7728394	3	NaN	6131796	6705355	NaN	NaN	NaN	NaN	
top	A-7777761	Source1	NaN	2021-01-26 16:16:13	2021-11-22 08:00:00	NaN	NaN	NaN	NaN	
freq	1	4325632	NaN	225	112	NaN	NaN	NaN	NaN	
mean	NaN	NaN	2.212384e+00	NaN	NaN	3.620119e+01	-9.470255e+01	3.626183e+01	-9.572557e+01	5.6184
std	NaN	NaN	4.875313e-01	NaN	NaN	5.076079e+00	1.739176e+01	5.272905e+00	1.810793e+01	1.7768
min	NaN	NaN	1.000000e+00	NaN	NaN	2.455480e+01	-1.246238e+02	2.456601e+01	-1.245457e+02	0.0000
25%	NaN	NaN	2.000000e+00	NaN	NaN	3.339963e+01	-1.172194e+02	3.346207e+01	-1.177543e+02	0.0000
50%	NaN	NaN	2.000000e+00	NaN	NaN	3.582397e+01	-8.776662e+01	3.618349e+01	-8.802789e+01	3.0000
75%	NaN	NaN	2.000000e+00	NaN	NaN	4.008496e+01	-8.035368e+01	4.017892e+01	-8.024709e+01	4.6400
max	NaN	NaN	4.000000e+00	NaN	NaN	4.900220e+01	-6.711317e+01	4.907500e+01	-6.710924e+01	4.4175

11 rows × 46 columns

```
df["Start_Time"] = pd.to_datetime(df["Start_Time"], errors="coerce")
df["hour"] = df["Start_Time"].dt.hour

df["hour"].value_counts().sort_index().plot(kind="bar", figsize=(10,4), color="#377eb8")
```

```
<Axes: xlabel='hour'>
```




```
df["Weather_Condition"] = df["Weather_Condition"].str.lower().str.strip()
df["Weather_Condition"].value_counts().head(20)
```

Weather_Condition	count
fair	2560802
mostly cloudy	1016195
cloudy	817082
clear	808743
partly cloudy	698972
overcast	382866
light rain	352957
scattered clouds	204829
light snow	128680
fog	99238
rain	84331
haze	76223
fair / windy	35671
heavy rain	32309
light drizzle	22684
thunder in the vicinity	17611
cloudy / windy	17035
t-storm	16810
mostly cloudy / windy	16508
snow	15537

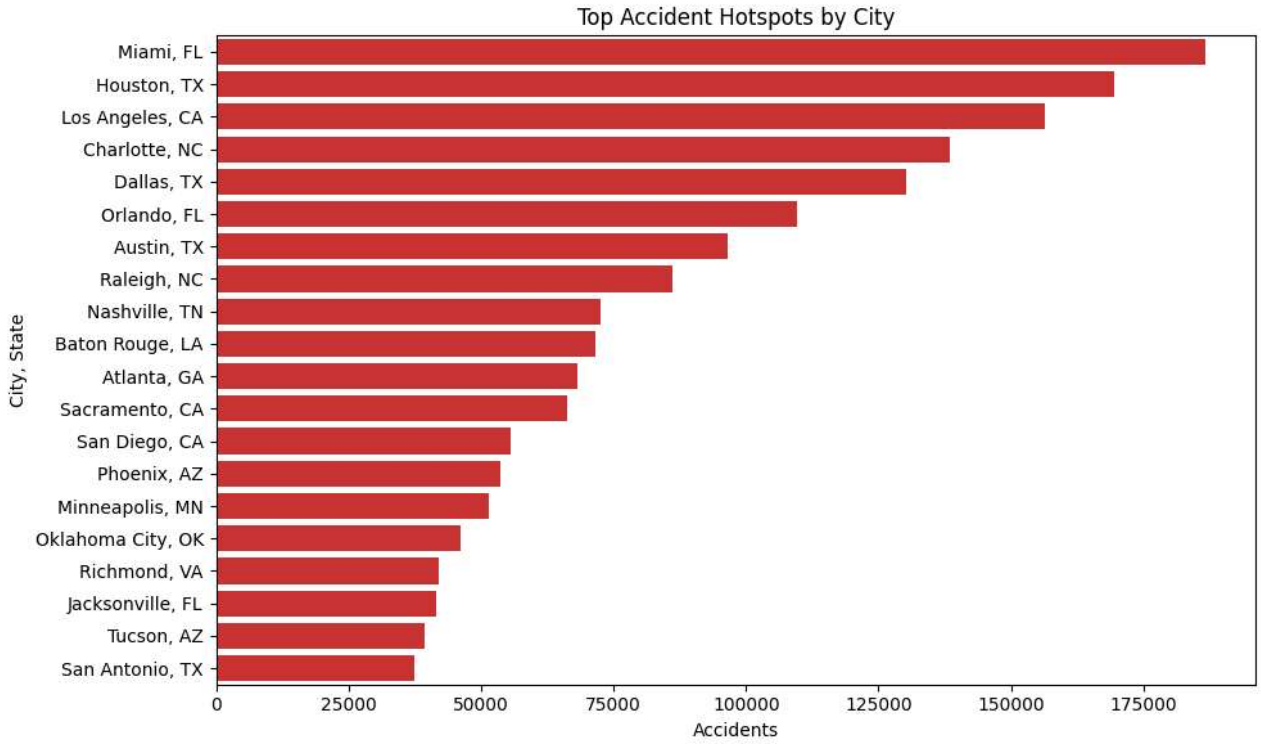
dtype: int64

```
import folium
from folium.plugins import HeatMap

# Check actual column names
print(df.columns[:10])  # look for 'Start_Lat' and 'Start_Lng'

# Use correct lat/lon columns
lat_col = "Start_Lat" if "Start_Lat" in df.columns else "Latitude"
lon_col = "Start_Lng" if "Start_Lng" in df.columns else "Longitude"

# Drop rows with missing coordinates
coords = df[[lat_col, lon_col]].dropna()

# Sample for speed
sample = coords.sample(n=50000, random_state=42)

# Create map
```

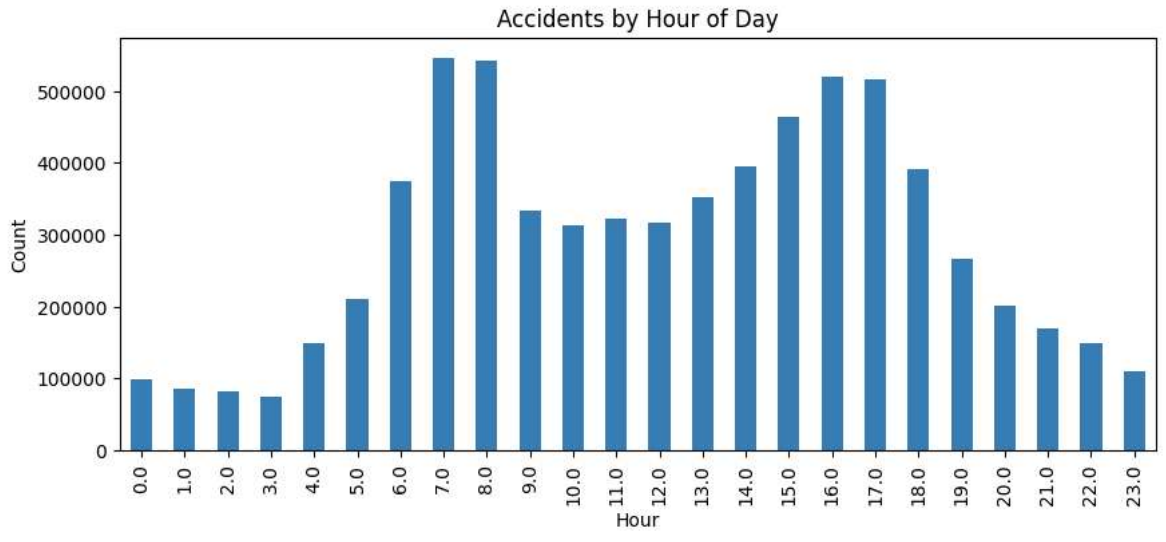
```
# Create map
m = folium.Map(location=[39.5, -98.35], zoom_start=4)  # US center
HeatMap(sample.values.tolist(), radius=6, blur=5).add_to(m)
m
```

```
Index(['ID', 'Source', 'Severity', 'Start_Time', 'End_Time', 'Start_Lat',
       'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)'],
      dtype='object')
```


```
city_counts = df.groupby(["State","City"]).size().reset_index(name="accidents")
top_cities = city_counts.sort_values("accidents", ascending=False).head(20)

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
sns.barplot(y=top_cities["City"] + ", " + top_cities["State"],
            x=top_cities["accidents"], color="#e41a1c")
plt.title("Top Accident Hotspots by City")
plt.xlabel("Accidents")
plt.ylabel("City, State")
plt.tight_layout()
plt.show()
```




```
df["Start_Time"] = pd.to_datetime(df["Start_Time"], errors="coerce")
df["hour"] = df["Start_Time"].dt.hour

df["hour"].value_counts().sort_index().plot(kind="bar", figsize=(10,4), color="#377eb8")
plt.title("Accidents by Hour of Day")
plt.xlabel("Hour")
plt.ylabel("Count")
plt.show()
```




```
df["Weather_Condition"] = df["Weather_Condition"].str.lower().str.strip()
top_weather = df["Weather_Condition"].value_counts().head(15)

plt.figure(figsize=(12,5))
sns.barplot(x=top_weather.index, y=top_weather.values, palette="Set2")
plt.xticks(rotation=45)
plt.title("Top Weather Conditions in Accidents")
plt.ylabel("Accident Count")
plt.show()
```

```
/tmp/ipython-input-986407839.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=top_weather.index, y=top_weather.values, palette="Set2")
```